

The systems being governed are in flux

1

Models

Swapped for a newer version, a cheaper provider or a fine-tune, and every system built on the old one inherits the change.

2

Platforms, systems & agents

Built on Foundry, Bedrock, Google or in-house orchestration, acting on business systems through tools, replaced in weeks.

3

AI inside the tools already in use

Copilots in productivity suites, assistants inside SaaS, GenAI in the delivery pipeline.
Adopted faster than any review cycle, often without anyone registering them.

Three clouds, four agent platforms, a hundred systems? The decisions about them cannot live in a single platform.

Three lines, one record

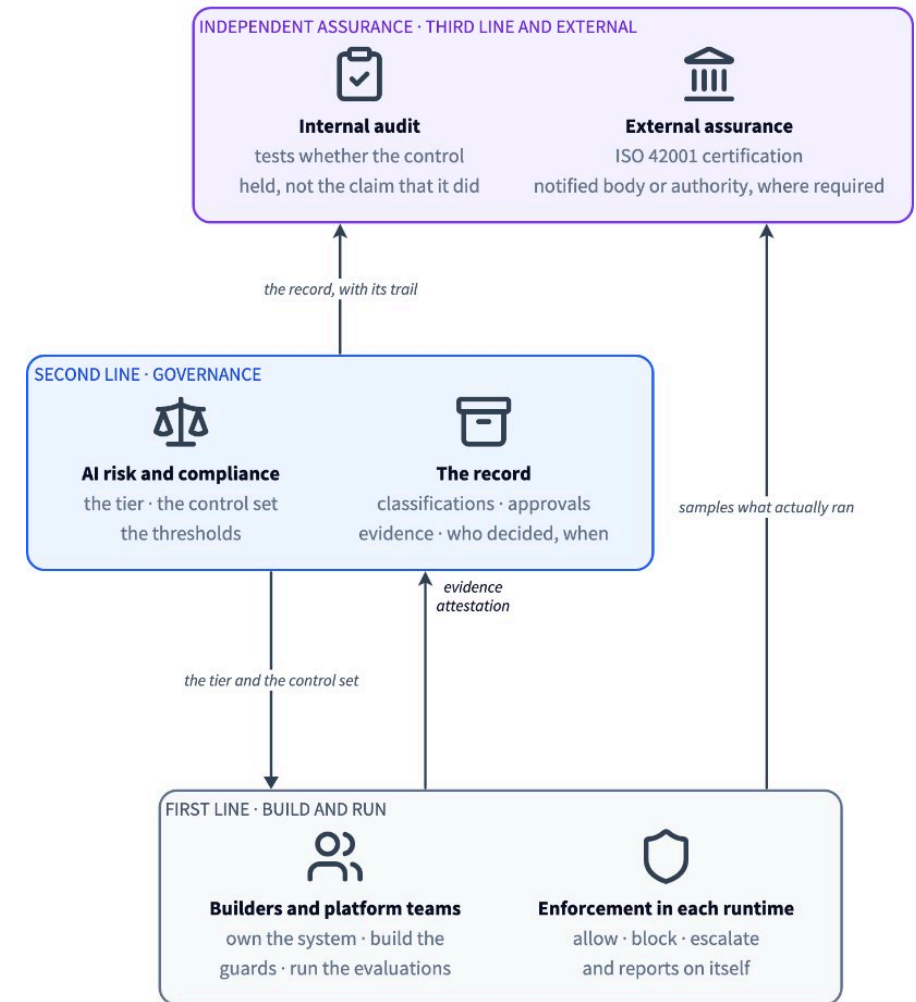
The model is not new. What is new is what sits in the first line: a platform that enforces its own controls and reports on itself.

The second line needs its own record. The first line emits the evidence. It does not hold the register, set the tier, or decide when a control is satisfied.

The third line tests that record against what actually ran, sampling the runtime directly. Neither internal audit nor an external assessor should have to ask the teams under audit to assemble their own evidence.

ISO/IEC 42001 asks for internal audit and management review.

The EU AI Act asks high-risk providers for a conformity assessment, in most cases their own, and the technical documentation behind it. Both assume the record exists.

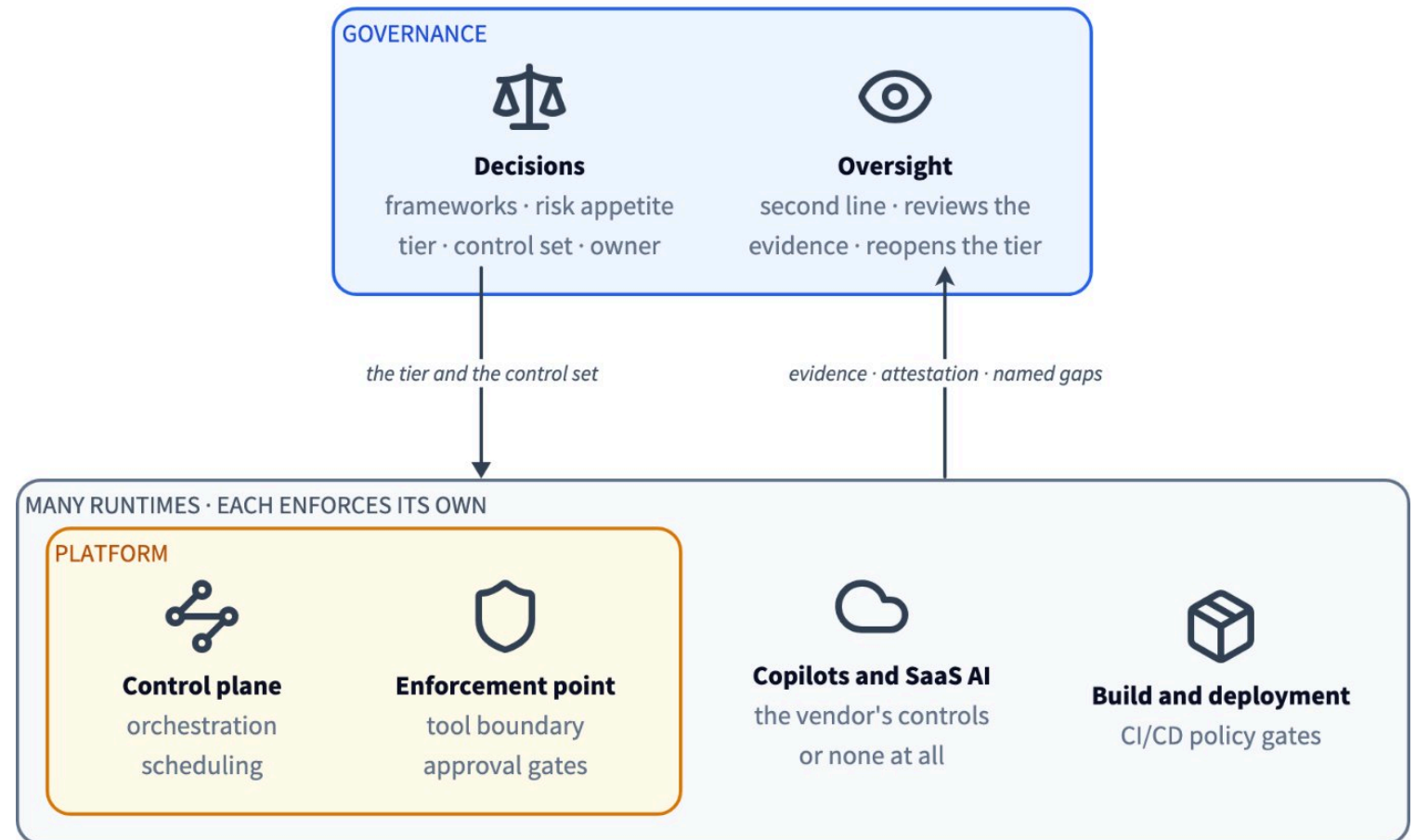


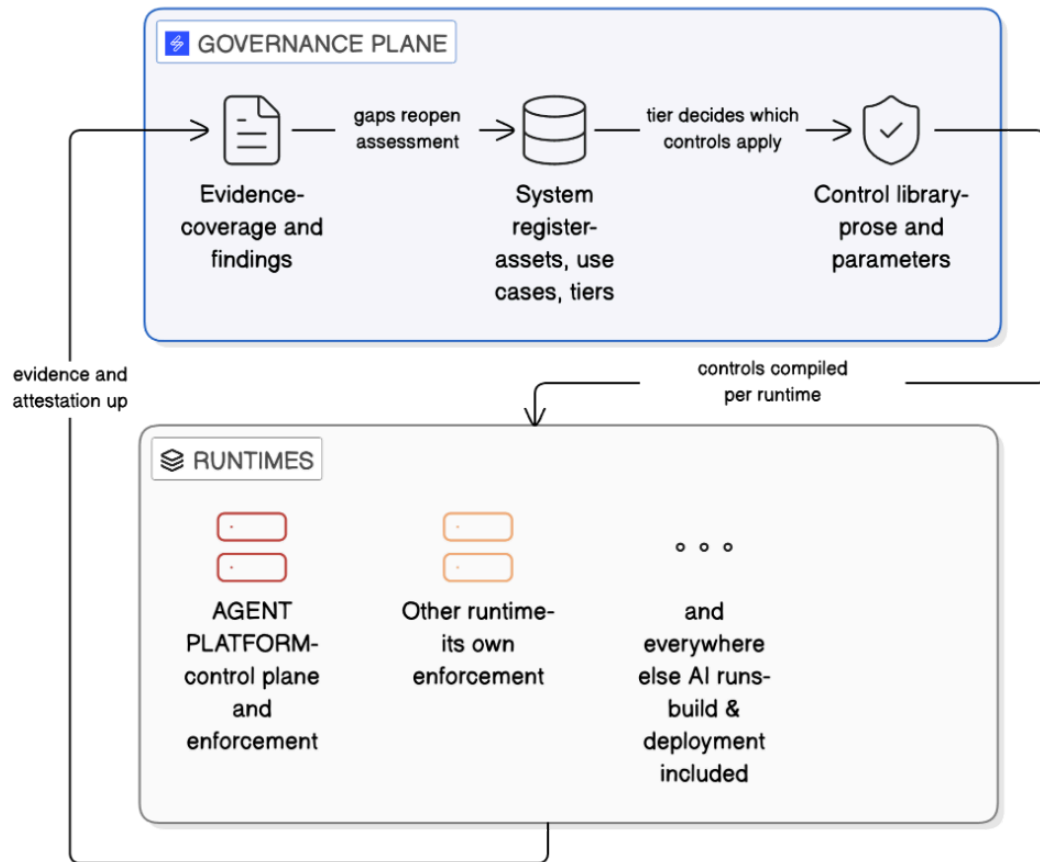
Governance is a separate role

The decisions are the organisation's: which frameworks apply, the risk appetite, the tier of each system, the controls it must satisfy, who is accountable. They have to survive a platform swap - or systems covering many - so they live above all of the platforms, in one place.

A runtime enforces what it runs and reports on itself. Oversight of that enforcement is a different accountability. The first line cannot be its own second.

The platform knows who ran a workflow. It does not know the system's purpose, its tier or its owner. Risk is contextual, so that is decided where the use case is.





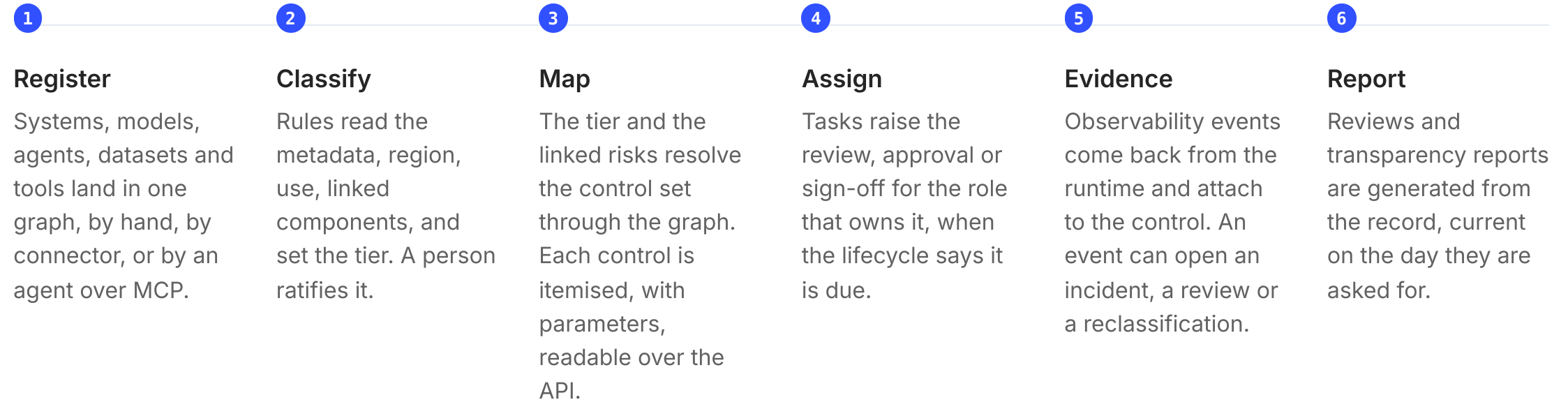
Two layers, one loop

Governance decides: the tier, the control set, the owner, the evidence required. It covers every runtime, including those with no enforcement of their own.

Enforcement applies the decision at the moment of action: allow, block, escalate. It lives in each runtime, it must be fast, and it is largely solved there already: Rego, Cedar, platform approval gates, model gateways.

Saidot conveys the decision down as itemised controls with parameters, and takes events back up as evidence. It does not sit in the hot path.

How a decision travels



Register, map and evidence run through the MCP servers as well, so a client's own agents can operate the loop.

Your governance model, configured

01 Your policies, as entities

Existing internal policies and their controls are modelled in the graph beside the **110+** policies in the library. They link to systems, risks and evidence like anything else. Not documents attached to a record.

02 Your process, as tasks

Lifecycle stages, approval gates, review cadence and the roles that own them are set per organisation. Governance becomes named work in a queue, not a policy nobody reads. In development.

03 Your automation, at your pace

Every rule is a workflow you switch on: classification, inheritance, control assignment, lifecycle approvals. What is not automated stays a task for a person.

Governance stays a gate. It stops being a queue.

Rules-based tiering

Properties decide the tier. A person ratifies it, because that one property scopes every obligation downstream.

Rules-based control profiles

The tier resolves its own control set. Nobody maps controls per system.

Lifecycle tasks

Reviews and sign-offs are raised when due, to the role that owns them.

Event-based triggers

Drift, an incident or a change reopens the assessment. Production keeps the record current.

Where this goes: the same loop at the scale of a bank's whole AI estate, with more of it automated and a person still deciding.